Lecture 3 — Advanced Greedy

> Regional Level · Prerequisites: Year Curriculum Weeks 4 (sorting, greedy basics), 13 (priority queues) · Estimated time: 2 hours

1. Overview

Greedy algorithms are short, which makes them dangerous: it is easy to type a plausible greedy in five minutes and collect two wrong answers. At regional level the greedy problems are rarely "take the largest first". They are problems where the correct greedy is *derived* from a proof, or where a naive greedy must be repaired with a heap.

This lecture covers three tools:

1. **Exchange arguments**, which turn into sorting comparators.
2. **Deadline scheduling** with a heap (including Moore–Hodgson and the backward scan).
3. **Regret greedy**: commit to choices eagerly, and when a constraint breaks, undo the worst choice so far.

The working rule for the whole track: *no proof sketch, no submission*. If you cannot sketch a proof in three minutes, write a brute force and stress test (Lecture 1) before submitting. Every program in these notes was verified against a brute force on hundreds of random small cases.

2. Exchange arguments

2.1 The template

To show that sorting by some key is optimal:

1. Consider any optimal solution. If it is not sorted by the key, it contains two **adjacent** elements `i, j` in the wrong order.
2. Swapping two adjacent elements changes only the terms involving `i` and `j`; everything before and after is unaffected.
3. Show the swap does not make the objective worse.
4. Repeated swaps (bubble sort) reach the sorted order without ever worsening the answer, so the sorted order is optimal.

Step 3 produces an inequality, and **that inequality is your comparator**. You do not need to guess the key; derive it.

2.2 Weighted completion time (Smith's rule)

There are `n` jobs; job `i` takes `t_i` time and has weight `w_i`. Jobs run one after another on one machine; `C_i` is the completion time of job `i`. Minimise `Σ w_i C_i`.

Take adjacent jobs `i` then `j` starting at time `T`. Their contribution is `w_i(T + t_i) + w_j(T + t_i + t_j)`. In the other order it is `w_j(T + t_j) + w_i(T + t_j + t_i)`. Subtracting, (i first) − (j first) = `w_j t_i − w_i t_j`. So `i` should go first when `t_i w_j < t_j w_i`, that is, when `t_i / w_i < t_j / w_j`.

Implement the comparator by **cross-multiplying integers**, never by comparing floating-point ratios. Floating-point ratios can compare unequal for mathematically equal values, and a comparator that is not a strict weak ordering is undefined behaviour in `std::sort` (it can even crash). For the same reason, use `<`, never `<=`. Check that the products fit: with values up to 10^9, the product is up to 10^{18}, which fits in `long long`; anything larger needs `__int128`.

The special case `w_i = 1` gives "shortest job first". CSES 1630 *Tasks and Deadlines* (reward `d_i − f_i`) reduces to it: the total reward is `Σ d_i − Σ f_i`, and `Σ f_i` is minimised by sorting by duration.

2.3 Splitting into classes: Codeforces 1203F1

Complete the Projects (easy version): you have rating `r`. Project `i` requires `r ≥ a_i` to start and changes the rating by `b_i` (possibly negative); the rating must never become negative. Can you complete all projects?

A single comparator does not work directly; first split into classes.

- **Gains** (`b_i ≥ 0`) never hurt, so do them all first, in increasing order of `a_i` (if the smallest requirement is not met, nothing is).
- **Losses** (`b_i < 0`): doing `i` then `j` requires `r ≥ max(a_i, a_j − b_i)` (ignoring the final non-negativity, which is order-independent). Doing `j` then `i` requires `r ≥ max(a_j, a_i − b_j)`. If `a_i + b_i ≥ a_j + b_j`, then `a_j − b_i ≤ a_i − b_j`, and `a_i ≤ a_i − b_j` since `b_j < 0`; so the first requirement is no larger. Hence sort losses by `a + b` **descending**.

The general lesson: if elements behave differently (gains vs. losses, positive vs. negative), split them and exchange within each class.

2.4 Minimising a maximum: Cow Acrobats

POJ 3045 — Cow Acrobats: cows with weight `w` and strength `s` form a tower. A cow's risk is the total weight above it minus its strength. Minimise the maximum risk.

For adjacent cows i (on top) and j with weight W above both, the risks are $W - s_i$ and $W + w_i - s_j$. Swapped, they are $W - s_j$ and $W + w_j - s_i$. If $w_i + s_i \le w_j + s_j$, both original risks are at most $W + w_j - s_i$, which is one of the swapped risks. So the cow with smaller $w + s$ goes on top. Exchange arguments work for max-objectives too; you compare the maximum of the pair instead of a sum.

3. Deadline scheduling

Scheduling with deadlines comes in several variants that look alike but need different methods:

Variant	Method
Unit-time jobs with profits and deadlines	backward scan with a max-heap, or sort by deadline with a min-heap of profits
Jobs with durations; maximise number on time	Moore–Hodgson
Reward $d - f$ per job	sort by duration
Durations and profits; maximise profit	sort by deadline, then knapsack DP over time

3.1 The backward scan: AtCoder ABC 137 D *Summer Vacation*

There are M days and n jobs. You may do one job per day; job i pays B_i exactly A_i days after you do it, and only payments received within M days count. Maximise income.

Forward in time, the set of usable jobs *shrinks* (a job with large A_i becomes unusable late), which is awkward. Backwards it *grows*: with k days left, a job is usable exactly when $A_i \le k$. So iterate $k = 1, \ldots, M$, push all jobs with $A_i \le k$ into a max-heap, and take the best one. An exchange argument shows that using the best available job for the most constrained day never hurts. Kattis `bank` (*Bank Queue*: each customer has an amount of cash and a time after which they leave, and you serve one customer per minute) has exactly the same structure.

3.2 Moore–Hodgson

Jobs have durations p_i and deadlines d_i; maximise the number of jobs finished on time. Process jobs by increasing deadline, adding each to a max-heap of chosen durations. If the total duration of chosen jobs exceeds the current deadline, remove the **longest** chosen job.

Why this works: after processing all jobs with deadline at most D, the heap contains a maximum-size feasible subset of those jobs, and among all maximum-size feasible subsets it has the smallest total duration. Adding a job increases the best count by at most one. If the

new set is infeasible, removing the longest job restores feasibility (chosen jobs are run in deadline order, and the total now fits) while keeping the count and minimising the total, which leaves the most room for future jobs.

4. Regret greedy

4.1 The idea

Take every option greedily. When a constraint is violated, undo the *worst* decision made so far; a heap remembers all decisions so the worst one is available in O(log n). The proof obligation is that undoing the worst decision restores an optimal state.

Typical signals: choose a subset while scanning in a fixed order, with a constraint on prefixes; buying and selling over time; a budget that may be exceeded.

4.2 Codeforces 1526C2 — Potions

Potions `a_1, …, a_n` (possibly negative) are offered in order. Health starts at 0 and must stay non-negative after each potion you drink. Maximise the number drunk.

Drink every potion. Whenever health becomes negative, "un-drink" the most negative potion drunk so far (the top of a min-heap). The count stays the same as before this step, and among sets of that size among the processed prefix, removing the most negative value leaves the maximal health, which can only help later. Note that un-drinking an earlier potion never breaks an earlier prefix: removing a negative value only raises all later prefix sums.

4.3 Codeforces 865D — Buy Low Sell High

Each day you know the share price; you may buy one share, sell one share, or do nothing. Maximise profit (you end with no shares).

Keep a min-heap of "buy options". On day `p`, if the cheapest option `x` is below `p`, sell: profit `p - x`. Then push `p` **twice**: once as an ordinary buy option for the future, and once as an *undo token*. If later a price `q > p` takes the token, the profit `q - p` added on top of `p - x` equals `q - x`: the sale moves from day `p` to day `q`, and day `p` becomes free. The token is the regret.

4.4 Merging and marginal gains

Huffman-style merging: CSES 1161 *Stick Divisions* asks for the cheapest way to cut a stick into given pieces where each cut costs the current length. Reversed, this is merging pieces with cost equal to the merged length; always merging the two smallest (min-heap) is optimal by the Huffman exchange argument.

Marginal gains: if `k` units of a resource are split among items, and each item's gain per extra unit is non-increasing (concave), repeatedly take the largest available marginal gain from a heap. Codeforces 1428E *Carrots for Rabbits* is a good regional-level example; the hard part is computing the marginal gain of one more cut.

5. When greedy fails

Before coding, try to break your greedy with three or four elements. Classic failures: coin change with denominations {1, 3, 4} and target 6 (greedy takes 4 + 1 + 1, optimal is 3 + 3); 0/1 knapsack by value density; weighted interval scheduling (needs DP with binary search, see Lecture 5). If small cases break it, switch to DP.

6. Reference implementations

6.1 Smith's rule (minimise $\Sigma w_i C_i$)

Input: `n`, then `n` lines `t w`.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<long long, long long>> job(n);        // (t, w)
    for (auto& [t, w] : job) cin >> t >> w;
    sort(job.begin(), job.end(), [](const auto& a, const auto& b) {
        return a.first * b.second < b.first * a.second; // t_a/w_a < t_b/w_b
    });
    long long time = 0, cost = 0;
    for (auto [t, w] : job) {
        time += t;
        cost += w * time;
    }
    cout << cost << '\n';
}
```

6.2 Moore–Hodgson

Input: `n`, then `n` lines `duration deadline`. Prints the maximum number of jobs finished on time.

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<long long, long long>> jobs(n);       // (deadline, duration)
    for (auto& [d, p] : jobs) cin >> p >> d;
    sort(jobs.begin(), jobs.end());
    priority_queue<long long> chosen;                 // durations, max-heap
    long long total = 0;
    for (auto [d, p] : jobs) {
        chosen.push(p);
        total += p;
        if (total > d) {
            total -= chosen.top();
            chosen.pop();
        }
    }
    cout << chosen.size() << '\n';
}
```

6.3 AtCoder ABC 137 D — Summer Vacation

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> job(n);                // (A, B)
    for (auto& [a, b] : job) cin >> a >> b;
    sort(job.begin(), job.end());
    priority_queue<int> avail;
    long long total = 0;
    int idx = 0;
    for (int k = 1; k <= m; k++) {                // k = days left
        while (idx < n && job[idx].first <= k) avail.push(job[idx++].second);
        if (!avail.empty()) { total += avail.top(); avail.pop(); }
    }
    cout << total << '\n';
}
```

6.4 Codeforces 1526C2 — Potions

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    priority_queue<long long, vector<long long>, greater<long long>> taken;
    long long health = 0;
    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        taken.push(a);
        health += a;
        if (health < 0) {               // undo the worst choice so far
            health -= taken.top();
            taken.pop();
        }
    }
    cout << taken.size() << '\n';
}
```

6.5 Codeforces 865D — Buy Low Sell High

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    long long profit = 0;
    for (int i = 0; i < n; i++) {
        long long p;
        cin >> p;
        if (!pq.empty() && pq.top() < p) {
            profit += p - pq.top();
            pq.pop();
            pq.push(p);                 // undo token
        }
        pq.push(p);                     // buy option
    }
    cout << profit << '\n';
}
```

7. Common mistakes

- **Comparators that are not strict weak orderings** (`<=`, inconsistent tie handling): undefined behaviour in `std::sort`.
- **Floating-point ratios** in comparators instead of exact cross-multiplication.
- **Overflow** in cross-multiplication or in accumulated costs.

- **Forgetting to split classes** (gains vs. losses) before applying an exchange argument.
- **Scanning forward when the available set shrinks**: try the backward scan.
- **Removing the wrong element** in Moore–Hodgson (remove the longest, not the current job).
- **Regret greedy without a proof**: it is easy to invent an "undo" rule that is wrong. Stress test.

8. Practice problems

- CSES 1630 — Tasks and Deadlines: https://cses.fi/problemset/task/1630
- CSES 1161 — Stick Divisions: https://cses.fi/problemset/task/1161
- CSES 1632 — Movie Festival II: https://cses.fi/problemset/task/1632
- AtCoder ABC 137 D — Summer Vacation: https://atcoder.jp/contests/abc137/tasks/abc137_d
- Kattis `bank` — Bank Queue: https://open.kattis.com/problems/bank
- Codeforces 1203F1 — Complete the Projects (easy version): https://codeforces.com/problemset/problem/1203/F1
- Codeforces 1526C2 — Potions (Hard Version): https://codeforces.com/problemset/problem/1526/C2
- Codeforces 865D — Buy Low Sell High: https://codeforces.com/problemset/problem/865/D
- Codeforces 1428E — Carrots for Rabbits: https://codeforces.com/problemset/problem/1428/E
- POJ 3045 — Cow Acrobats: http://poj.org/problem?id=3045

9. Further reading

- Jon Kleinberg and Éva Tardos, *Algorithm Design*, Chapter 4 (greedy algorithms, exchange arguments, scheduling to minimise lateness).
- W. E. Smith, "Various optimizers for single-stage production" (1956) — the original weighted completion time rule.
- J. M. Moore, "An n job, one machine sequencing algorithm for minimizing the number of late jobs" (1968).
- Codeforces blog posts on "regret greedy" / "反悔贪心" (search the term), with many more examples of undo tokens.